

5 Variantes de Teste Laboratorial - Estruturas de Dados

TESTE 1

1 – Análise de Complexidade

Considere o seguinte código e indique a complexidade em função de N. Apresente uma breve justificação:

```
java
for(int j=N; j>1; j/=3)
    for(int i=0; i<N*N; i++)
        soma++;
```

2 – Método tamanhoUnião

Assuma que dispõe de um método `(int pesquisa(int m[], int valor))` que efectua uma pesquisa binária. Este método devolve a posição em que o valor procurado se encontra, ou então um valor negativo (-X) caso este não esteja no array indicado. O valor de $\text{abs}(X+1)$ indica uma posição em que o valor procurado poderia ser inserido para preservar a ordem.

Construa um método `(int tamanhoUnião(int m[], int n[]))` que devolve a dimensão de um array que representa a união de m e n (sem repetições). Cada um dos arrays m e n estão ordenados e têm elementos repetidos. O método deve ter desempenho de ordem $O(N \cdot \log(M))$, onde N e M são a dimensão dos arrays n e m, respectivamente.

Por exemplo, se $n=\{1,2,2,5,8\}$ e $m=\{1,1,3,5,6,7\}$, então o método deve devolver 7 (correspondentes aos números 1,2,3,5,6,7,8).

3 – Método g - Remoção eficiente

Construa um método g que recebe dois parâmetros:

- um parâmetro **primeiro** que é uma List de um determinado tipo genérico T
- um parâmetro **segundo** que é uma lista que contém valores que podem ser comparados com os valores da primeira lista.

O método deve remover de **primeiro** todos os valores **maiores** do que todos os valores que se encontram em **segundo**. Deve assegurar-se que o método é tão eficiente quanto o possível, sem utilizar espaço adicional, independentemente do tipo de lista que é recebida (ArrayList ou LinkedList).

4 – Classe MinhaLista

Crie a classe iterável **MinhaLista**, que armazena uma lista de inteiros, inicializada a partir do seu construtor. Não é possível alterar esta lista após a sua inicialização. Construa um iterador que percorre apenas os números **pares** dessa lista, em ordem inversa.

Inclua suporte para todas as exceções que podem ser lançadas (deve ignorar aquelas que não fazem sentido para esta estrutura de dados).

TESTE 2

1 – Análise de Complexidade

Considere o seguinte código e indique a complexidade em função de N. Apresente uma breve justificação:

```
java
for(int j=0; j<N; j++)
    for(int i=1; i<N; i*=3)
        for(int k=0; k<j; k++)
            soma++;
```

2 – Método contarDiferentes

Assuma que dispõe de um método `int pesquisa(int m[], int valor)` que efectua uma pesquisa binária. Este método devolve a posição em que o valor procurado se encontra, ou então um valor negativo (-X) caso este não esteja no array indicado.

Construa um método `int contarDiferentes(int m[], int n[])` que devolve quantos elementos de **n não se encontram** em m. Cada um dos arrays m e n estão ordenados e podem ter elementos repetidos. O método deve ter desempenho de ordem $O(N \cdot \log(M))$, onde N e M são a dimensão dos arrays n e m, respectivamente.

Por exemplo, se $n=\{1,2,2,5,9,9\}$ e $m=\{1,3,5,6,7\}$, então o método deve devolver 3 (correspondentes a 2, 9 e 9).

3 – Método h - Duplicar elementos

Construa um método h que recebe dois parâmetros:

- um parâmetro **lista** que é uma List de um determinado tipo genérico T
- um parâmetro **valores** que é uma lista que contém valores que podem ser comparados com os valores da primeira lista.

O método deve **duplicar** em **lista** todos os elementos que são **iguais** a algum valor em **valores**. Deve assegurar-se que o método é tão eficiente quanto o possível, sem utilizar espaço adicional significativo, independentemente do tipo de lista que é recebida (ArrayList ou LinkedList).

4 – Classe PalavraIterável

Crie a classe iterável **PalavraIterável**, que armazena uma string, inicializada a partir do seu construtor. Não é possível alterar esta string após a sua inicialização. Construa um iterador que percorre cada **palavra** dessa string (separadas por espaços), convertendo-as para **maiúsculas**.

Por exemplo, se a string for "Olá mundo feliz", o iterador percorre "OLÁ", "MUNDO", "FELIZ".

Inclua suporte para todas as exceções que podem ser lançadas.

TESTE 3

1 – Análise de Complexidade

Considere o seguinte código e indique a complexidade em função de N. Apresente uma breve justificação:

```
java
for(int j=1; j<N; j++)
    for(int i=j; i>0; i/=2)
        soma++;
```

2 – Método elementosComuns

Assuma que dispõe de um método `(int pesquisa(int m[], int valor))` que efectua uma pesquisa binária. Este método devolve a posição em que o valor procurado se encontra, ou então um valor negativo (-X) caso este não esteja no array indicado.

Construa um método `(int[] elementosComuns(int m[], int n[]))` que devolve um novo array contendo todos os elementos de n que **aparecem pelo menos uma vez** em m (sem duplicações no resultado). Cada um dos arrays m e n estão ordenados e podem ter elementos repetidos. O método deve ter desempenho de ordem $O(N * \log(M) + N)$, onde N e M são a dimensão dos arrays n e m, respectivamente.

Por exemplo, se $n = \{1, 1, 3, 3, 5, 7\}$ e $m = \{2, 3, 3, 4, 5\}$, então o método deve devolver $\{3, 5\}$.

3 – Método k - Manter maiores

Construa um método k que recebe dois parâmetros:

- um parâmetro **lista** que é uma List de um determinado tipo genérico T
- um parâmetro **limite** que é um valor do tipo T

O método deve remover de **lista** todos os valores **menores ou iguais** a **limite**. Deve assegurar-se que o método é tão eficiente quanto o possível, sem utilizar espaço adicional, independentemente do tipo de lista que é recebida (ArrayList ou LinkedList).

4 – Classe ConjuntoIterável

Crie a classe iterável **ConjuntoIterável**, que armazena um array de inteiros, inicializado a partir do seu construtor. Não é possível alterar este array após a sua inicialização. Construa um iterador que percorre cada elemento **único** desse array (ignorando duplicados), em ordem crescente.

Por exemplo, se o array for {5,2,8,2,1,5,3}, o iterador percorre 1,2,3,5,8.

Inclua suporte para todas as exceções que podem ser lançadas.

TESTE 4

1 – Análise de Complexidade

Considere o seguinte código e indique a complexidade em função de N. Apresente uma breve justificação:

```
java
for(int j=0; j<N; j++)
    for(int i=0; i<j*j; i++)
        soma++;
```

2 – Método primeiraOcorrencia

Assuma que dispõe de um método `int pesquisa(int m[], int valor)` que efectua uma pesquisa binária. Este método devolve a posição em que o valor procurado se encontra, ou então um valor negativo (-X) caso este não esteja no array indicado.

Construa um método `int primeiraOcorrencia(int m[], int n[])` que devolve o **índice** do primeiro elemento de n que também se encontra em m. Se nenhum elemento de n estiver em m, devolve -1. Cada um dos arrays m e n estão ordenados e podem ter elementos repetidos. O método deve ter desempenho de ordem **$O(N \cdot \log(M))$** , onde N e M são a dimensão dos arrays n e m, respectivamente.

Por exemplo, se $n=\{1,2,5,7,9\}$ e $m=\{3,5,6,8,9\}$, então o método deve devolver 2 (índice do número 5 em n).

3 – Método m - Filtrar intervalo

Construa um método m que recebe três parâmetros:

- um parâmetro **lista** que é uma List de um determinado tipo genérico T
- um parâmetro **minimo** que é um valor do tipo T
- um parâmetro **maximo** que é um valor do tipo T

O método deve remover de **lista** todos os valores que **não estão** no intervalo [minimo, maximo] (inclusive). Deve assegurar-se que o método é tão eficiente quanto o possível, sem utilizar espaço adicional, independentemente do tipo de lista que é recebida (ArrayList ou LinkedList).

4 – Classe SequenciaNumérica

Crie a classe iterável **SequenciaNumérica**, que armazena dois inteiros (inicio e fim), inicializados a partir do seu construtor. Construa um iterador que percorre todos os números **ímpares** entre inicio e fim (inclusive), em ordem crescente.

Por exemplo, se inicio=3 e fim=12, o iterador percorre 3,5,7,9,11.

Inclua suporte para todas as exceções que podem ser lançadas.

TESTE 5

1 – Análise de Complexidade

Considere o seguinte código e indique a complexidade em função de N. Apresente uma breve justificação:

```
java
for(int j=N; j>0; j--)
    for(int i=0; i<N; i+=j)
        soma++;
```

2 – Método substituirComuns

Assuma que dispõe de um método `(int pesquisa(int m[], int valor))` que efectua uma pesquisa binária. Este método devolve a posição em que o valor procurado se encontra, ou então um valor negativo (-X) caso este não esteja no array indicado.

Construa um método `(void substituirComuns(int m[], int n[], int novoValor))` que substitui em n todos os valores que também se encontram em m pelo valor **novoValor**. Cada um dos arrays m e n estão ordenados e podem ter

elementos repetidos. O método deve ter desempenho de ordem $O(N \cdot \log(M))$, onde N e M são a dimensão dos arrays n e m, respectivamente.

Por exemplo, se $n=\{1,2,2,5,8\}$ e $m=\{2,3,5,6\}$, após a execução n ficará $\{1,-1,-1,-1,8\}$ (assumindo novoValor=-1).

3 – Método p - Remover intervalos

Construa um método p que recebe dois parâmetros:

- um parâmetro **lista** que é uma List de um determinado tipo genérico T
- um parâmetro **referencias** que é uma lista de valores do tipo T

O método deve remover de **lista** todos os valores que estão **entre quaisquer dois valores consecutivos** em **referencias**. Deve assegurar-se que o método é tão eficiente quanto o possível, sem utilizar espaço adicional significativo, independentemente do tipo de lista que é recebida (ArrayList ou LinkedList).

4 – Classe TextoInvertido

Crie a classe iterável **TextoInvertido**, que armazena uma string, inicializada a partir do seu construtor. Não é possível alterar esta string após a sua inicialização. Construa um iterador que percorre cada **Character** dessa string em **ordem inversa**, ignorando todos os **dígitos** (0-9).

Por exemplo, se a string for "abc123xyz", o iterador percorre 'z','y','x','c','b','a'.

Inclua suporte para todas as exceções que podem ser lançadas.